

Лабораторная работа №5

Линейное программирование и транспортная задача

Дисциплина: Методы оптимизации

Цель работы: Комплексное изучение и практическое освоение методов линейного программирования и их применения к решению транспортной задачи с использованием современных вычислительных инструментов на языке Python.

Теоретическая часть

1. Основы линейного программирования

1.1. Постановка задачи линейного программирования

Линейное программирование (ЛП) — математическая дисциплина, посвященная решению задач оптимизации линейной функции на множестве, задаваемом системой линейных уравнений и неравенств.

Общая форма задачи ЛП:

Целевая функция:

$$f(\mathbf{x}) = c_1x_1 + c_2x_2 + \dots + c_nx_n \rightarrow \text{extr}$$

Ограничения:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \leq b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \leq b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \leq b_m \\ x_1, x_2, \dots, x_n \geq 0 \end{cases}$$

В матричной форме:

$$\begin{cases} \mathbf{c}^T \mathbf{x} \rightarrow \max(\text{или } \min) \\ A\mathbf{x} \leq \mathbf{b} \\ \mathbf{x} \geq \mathbf{0} \end{cases}$$

где:

- $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ — вектор переменных
- $\mathbf{c} = (c_1, c_2, \dots, c_n)^T$ — вектор коэффициентов целевой функции
- $A = [a_{ij}]_{m \times n}$ — матрица коэффициентов ограничений
- $\mathbf{b} = (b_1, b_2, \dots, b_m)^T$ — вектор правых частей ограничений

1.2. Каноническая и стандартная формы

Каноническая форма (для минимизации):

$$\begin{cases} \mathbf{c}^T \mathbf{x} \rightarrow \min \\ A\mathbf{x} = \mathbf{b} \\ \mathbf{x} \geq \mathbf{0} \end{cases}$$

Стандартная форма (для максимизации):

$$\begin{cases} \mathbf{c}^T \mathbf{x} \rightarrow \max \\ A\mathbf{x} \leq \mathbf{b} \\ \mathbf{x} \geq \mathbf{0} \end{cases}$$

Преобразование между формами:

1. Преобразование неравенств в равенства с помощью переменных slack:

$$\sum a_{ij}x_j \leq b_i \Rightarrow \sum a_{ij}x_j + s_i = b_i, \quad s_i \geq 0$$

2. Замена переменных:

$$x_j \text{ без ограничения знака} \Rightarrow x_j = x_j^+ - x_j^-, \quad x_j^+, x_j^- \geq 0$$

3. Изменение направления оптимизации:

$$\max f(\mathbf{x}) = -\min[-f(\mathbf{x})]$$

1.3. Геометрическая интерпретация

Множество допустимых решений образует выпуклый многогранник в $\mathbb{R}^n$. Для задачи с n переменными и m ограничениями:

- Каждое ограничение определяет полупространство
- Пересечение полупространств образует многогранник решений
- Оптимальное решение достигается в вершине многогранника (если оно существует)
- Линии уровня целевой функции — параллельные гиперплоскости

Теорема о существовании решения:

Если задача ЛП имеет конечное оптимальное решение, то оно достигается хотя бы в одной вершине многогранника допустимых решений.

1.4 Симплекс-метод

Основные понятия и определения

Исходная задача линейного программирования

Рассмотрим задачу в **стандартной форме**:

$$\begin{cases} z = c_1x_1 + c_2x_2 + \dots + c_nx_n \rightarrow \max \\ a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \leq b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \leq b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \leq b_m \\ x_1, x_2, \dots, x_n \geq 0 \end{cases}$$

Введение slack-переменных

Для преобразования к **канонической форме** вводим slack-переменные $s_i \geq 0$:

$$\begin{cases} z = c_1x_1 + c_2x_2 + \dots + c_nx_n \rightarrow \max \\ a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n + s_1 = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n + s_2 = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n + s_m = b_m \\ x_1, x_2, \dots, x_n \geq 0, \quad s_1, s_2, \dots, s_m \geq 0 \end{cases}$$

Матричная запись

Расширенная система:

$$\begin{cases} z = \mathbf{c}^T \mathbf{x} \rightarrow \max \\ A\mathbf{x} + I\mathbf{s} = \mathbf{b} \\ \mathbf{x} \geq \mathbf{0}, \quad \mathbf{s} \geq \mathbf{0} \end{cases}$$

где:

- $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ - вектор основных переменных
- $\mathbf{s} = (s_1, s_2, \dots, s_m)^T$ - вектор slack-переменных
- $A = [a_{ij}]_{m \times n}$ - матрица коэффициентов
- I - единичная матрица размера $m \times m$
- $\mathbf{b} = (b_1, b_2, \dots, b_m)^T$ - вектор правых частей
- $\mathbf{c} = (c_1, c_2, \dots, c_n)^T$ - вектор коэффициентов целевой функции

Базисные и небазисные переменные

Определения

Базисные переменные (basic variables) - m переменных, которые принимают ненулевые значения и соответствуют линейно независимым столбцам матрицы ограничений.

Небазисные переменные (non-basic variables) - остальные n переменных, которые полагаются равными нулю.

Базисное решение

Если разделить переменные на базисные $\mathbf{x}_B$ и небазисные $\mathbf{x}_N$, то система примет вид:

$$B\mathbf{x}_B + N\mathbf{x}_N = \mathbf{b}$$

где:

- B - невырожденная $m \times m$ матрица (базис)
- N - $m \times n$ матрица небазисных столбцов

При $\mathbf{x}_N = \mathbf{0}$ получаем **базисное решение**:

$$\mathbf{x}_B = B^{-1}\mathbf{b}$$

Симплекс-таблица

Общий вид симплекс-таблицы

Базис	x_1	x_2	$\dots$	x_n	s_1	s_2	$\dots$	s_m	Решение
s_1	a_{11}	a_{12}	$\dots$	a_{1n}	1	0	$\dots$	0	b_1
s_2	a_{21}	a_{22}	$\dots$	a_{2n}	0	1	$\dots$	0	b_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
s_m	a_{m1}	a_{m2}	$\dots$	a_{mn}	0	0	$\dots$	1	b_m
z	$-c_1$	$-c_2$	$\dots$	$-c_n$	0	0	$\dots$	0	0

Компактная форма симплекс-таблицы

После k итераций таблица имеет вид:

Базис	$\mathbf{x}_N$	Решение
$\mathbf{x}_B$	$Y = B^{-1}N$	$\mathbf{b}' = B^{-1}\mathbf{b}$
z	$\mathbf{c}_N^T - \mathbf{c}_B^T B^{-1}N$	$\mathbf{c}_B^T B^{-1}\mathbf{b}$

где:

- $\mathbf{b}'$ - текущие значения базисных переменных
- Y - коэффициенты при небазисных переменных
- $\mathbf{c}_N^T - \mathbf{c}_B^T B^{-1}N$ - симплекс-разности

Критерии оптимальности и правила симплекс-метода

Симплекс-разности (reduced costs)

Для каждой небазисной переменной x_j вычисляется **симплекс-разность**:

$$\Delta_j = c_j - \mathbf{c}_B^T B^{-1} \mathbf{A}_j$$

где $\mathbf{A}_j$ - j -й столбец исходной матрицы A .

Критерий оптимальности:

- Для задачи **максимизации**: если все $\Delta_j \leq 0$, решение оптимально ✓
- Для задачи **минимизации**: если все $\Delta_j \geq 0$, решение оптимально ✓

Правило выбора вводимой переменной

Для максимизации:

$$k = \arg \max \{ \Delta_j : \Delta_j > 0 \}$$

Для минимизации:

$$k = \arg \min \{ \Delta_j : \Delta_j < 0 \}$$

Переменная x_k вводится в базис.

Правило выбора исключаемой переменной

Для вводимой переменной x_k вычисляем:

$$\theta_i = \frac{b'_i}{y_{ik}} \quad \text{для всех } i \text{ таких, что } y_{ik} > 0$$

Исключаемая переменная определяется по **минимальному отношению**:

$$r = \arg \min \{ \theta_i : y_{ik} > 0 \}$$

Переменная, соответствующая строке r , исключается из базиса.

Правило пересчета симплекс-таблицы

Жорданово исключение:

1. **Ведущий элемент:** y_{rk}
2. **Ведущая строка** (строка r):

$$y'_{rj} = \frac{y_{rj}}{y_{rk}}, \quad b'_r = \frac{b_r}{y_{rk}}$$

3. **Остальные строки** ($i \neq r$):

$$y'_{ij} = y_{ij} - \frac{y_{ik} \cdot y_{rj}}{y_{rk}}, \quad b'_i = b_i - \frac{y_{ik} \cdot b_r}{y_{rk}}$$

4. **Строка целевой функции:**

$$\Delta'_j = \Delta_j - \frac{\Delta_k \cdot y_{rj}}{y_{rk}}, \quad z' = z + \frac{\Delta_k \cdot b_r}{y_{rk}}$$

Алгоритм симплекс-метода

Пошаговый алгоритм

1. **Инициализация:**
 - Привести задачу к канонической форме
 - Найти начальное базисное решение
 - Построить начальную симплекс-таблицу
2. **Проверка оптимальности:**
 - Вычислить симплекс-разности Δ_j для всех небазисных переменных
 - Если критерий оптимальности выполнен $\rightarrow$ СТОП, решение найдено
3. **Выбор вводимой переменной:**
 - Выбрать переменную с наибольшей положительной симплекс-разностью (для максимизации)
4. **Проверка неограниченности:**
 - Если все $y_{ik} \leq 0 \rightarrow$ СТОП, задача неограничена

5. Выбор исключаемой переменной:

- Вычислить отношения $\theta_i = b'_i / y_{ik}$ для $y_{ik} > 0$
- Выбрать переменную с минимальным θ_i

6. Пересчет таблицы:

- Выполнить шаг жорданова исключения
- Перейти к шагу 2

Нахождение начального базиса

Метод искусственного базиса (Big-M method):

1. Добавляем искусственные переменные $w_i \geq 0$:

$$\sum a_{ij}x_j + s_i + w_i = b_i$$

2. Модифицируем целевую функцию:

$$z = \mathbf{c}^T \mathbf{x} - M \sum w_i \rightarrow \max$$

где M - достаточно большое число

3. Решаем задачу симплекс-методом

4. Если в оптимальном решении $w_i = 0$ для всех i , найдено допустимое решение исходной задачи

Математические основы

Теорема о представлении

Любое допустимое решение $\mathbf{x}$ можно представить в виде:

$$\mathbf{x} = \sum_{i=1}^p \lambda_i \mathbf{v}_i + \sum_{j=1}^q \mu_j \mathbf{d}_j$$

где:

- $\mathbf{v}_i$ - вершины многогранника
- $\mathbf{d}_j$ - направления неограниченности
- $\lambda_i \geq 0, \sum \lambda_i = 1, \mu_j \geq 0$

Теорема об оптимальности

Если задача линейного программирования имеет оптимальное решение, то существует **вершина** многогранника допустимых решений, в которой достигается оптимум.

Конечность алгоритма

При отсутствии **вырождения** (когда $b'_i > 0$ для всех базисных переменных) симплекс-метод сходится за конечное число итераций.

Особые случаи

Вырождение

Когда $b'_i = 0$ для некоторой базисной переменной, может возникнуть **зацикливание**. Методы борьбы:

- Правило Бленда
- Возмущение правых частей
- Лексикографический порядок

Неограниченность

Если для вводимой переменной x_k все $y_{ik} \leq 0$, то целевая функция неограничена.

Альтернативные оптимумы

Если для некоторой небазисной переменной с нулевой симплекс-разностью существует положительный y_{ik} , то существуют альтернативные оптимальные решения.

Модификации симплекс-метода

Revised Simplex Method

Работает непосредственно с исходными матрицами, не храня всю симплекс-таблицу:

1. Вычисляем $\mathbf{w}^T = \mathbf{c}_B^T B^{-1}$ (симплекс-множители)
2. Вычисляем $\Delta_j = c_j - \mathbf{w}^T \mathbf{A}_j$
3. Если решение не оптимально, находим вводимую переменную
4. Вычисляем $\mathbf{y}_k = B^{-1} \mathbf{A}_k$
5. Находим исключаемую переменную
6. Обновляем обратную матрицу B^{-1}

Dual Simplex Method

Решает двойственную задачу, полезен при добавлении ограничений.

Пример вычислений

Рассмотрим задачу:

$$\begin{cases} z = 3x_1 + 2x_2 \rightarrow \max \\ x_1 + x_2 \leq 4 \\ 2x_1 + x_2 \leq 5 \\ x_1, x_2 \geq 0 \end{cases}$$

Шаг 1: Вводим slack-переменные:

$$\begin{cases} z = 3x_1 + 2x_2 \rightarrow \max \\ x_1 + x_2 + s_1 = 4 \\ 2x_1 + x_2 + s_2 = 5 \\ x_1, x_2, s_1, s_2 \geq 0 \end{cases}$$

Начальная таблица:

Базис	x_1	x_2	s_1	s_2	Решение
s_1	1	1	1	0	4
s_2	2	1	0	1	5
z	-3	-2	0	0	0

Итерация 1:

- Вводим x_1 (наибольшая положительная симплекс-разность 3)
- Исключаем s_2 ($\min\{4/1, 5/2\} = 2.5$)

После пересчета получаем оптимальное решение: $x_1 = 2.5, x_2 = 0, z = 7.5$

Данное подробное описание охватывает все аспекты симплекс-метода от основных понятий до практической реализации.

2. Транспортная задача

Транспортная задача — специальный случай задачи линейного программирования, имеющий специфическую структуру матрицы ограничений.

Исходные данные:

- m — количество поставщиков
- n — количество потребителей
- $a_i > 0$ — запас груза у i -го поставщика, $i = 1, 2, \dots, m$
- $b_j > 0$ — потребность j -го потребителя, $j = 1, 2, \dots, n$
- $c_{ij} \geq 0$ — стоимость перевозки единицы груза от i -го поставщика к j -му потребителю

Переменные решения:

- $x_{ij} \geq 0$ — количество груза, перевозимого от поставщика i к потребителю j

Математическая модель

Целевая функция (минимизация общей стоимости перевозок):

$$Z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \rightarrow \min$$

Ограничения поставщиков (весь запас должен быть вывезен):

$$\sum_{j=1}^n x_{ij} = a_i, \quad i = 1, 2, \dots, m$$

Ограничения потребителей (все потребности должны быть удовлетворены):

$$\sum_{i=1}^m x_{ij} = b_j, \quad j = 1, 2, \dots, n$$

Условия неотрицательности:

$$x_{ij} \geq 0, \quad i = 1, 2, \dots, m; \quad j = 1, 2, \dots, n$$

Условие баланса

Транспортная задача имеет решение тогда и только тогда, когда выполняется **условие баланса**:

$$\sum_{i=1}^m a_i = \sum_{j=1}^n b_j$$

Если $\sum a_i > \sum b_j$, вводим фиктивного потребителя с потребностью:

$$b_{n+1} = \sum_{i=1}^m a_i - \sum_{j=1}^n b_j$$

и стоимостями $c_{i,n+1} = 0$ для всех i .

Если $\sum a_i < \sum b_j$, вводим фиктивного поставщика с запасом:

$$a_{m+1} = \sum_{j=1}^n b_j - \sum_{i=1}^m a_i$$

и стоимостями $c_{m+1,j} = 0$ для всех j .

Свойства транспортной задачи

Ранг системы ограничений:

Система имеет $m + n$ уравнений, но только $m + n - 1$ линейно независимых ограничений, так как сумма всех уравнений поставщиков равна сумме всех уравнений потребителей.

Базисное решение:

Базисное решение содержит не более $m + n - 1$ положительных компонент x_{ij} . Остальные переменные равны нулю.

Теорема о существовании решения:

Если выполнено условие баланса и все $a_i > 0$, $b_j > 0$, то транспортная задача имеет допустимое решение.

Методы нахождения начального опорного решения

Метод северо-западного угла

Алгоритм:

1. Начинаем с левой верхней ячейки $(1, 1)$
2. $x_{11} = \min(a_1, b_1)$
3. Если $a_1 > b_1$, то $a_1 := a_1 - b_1$, переходим к ячейке $(1, 2)$
4. Если $a_1 < b_1$, то $b_1 := b_1 - a_1$, переходим к ячейке $(2, 1)$
5. Если $a_1 = b_1$, переходим к ячейке $(2, 2)$
6. Повторяем до заполнения всех запасов и потребностей

Математическая запись:

На каждом шаге для текущей ячейки (i, j) :

$$x_{ij} = \min(a_i, b_j)$$

$$a_i := a_i - x_{ij}, \quad b_j := b_j - x_{ij}$$

Метод минимального элемента

Алгоритм:

1. Находим ячейку с минимальной стоимостью c_{ij}
2. $x_{ij} = \min(a_i, b_j)$
3. Корректируем запасы и потребности
4. Исключаем заполненного поставщика или потребителя
5. Повторяем для оставшихся ячеек

Формализация:

На k -м шаге:

$$(i_k, j_k) = \arg \min \{c_{ij} : i \in I_k, j \in J_k\}$$

где I_k — множество неиспользованных поставщиков, J_k — множество неудовлетворенных потребителей.

Метод Фогеля (аппроксимации)

Алгоритм:

1. Для каждой строки и столбца вычисляем разность между двумя минимальными стоимостями
2. Выбираем строку или столбец с максимальной разностью
3. В выбранной строке/столбце назначаем перевозку в ячейку с минимальной стоимостью
4. Корректируем запасы и потребности
5. Повторяем до заполнения всех ячеек

Формулы:

Для строки i : $R_i = c_{i,j_2} - c_{i,j_1}$, где $c_{i,j_1} \leq c_{i,j_2}$ — две минимальные стоимости в строке

Для столбца j : $C_j = c_{i_2,j} - c_{i_1,j}$, где $c_{i_1,j} \leq c_{i_2,j}$ — две минимальные стоимости в столбце

Метод потенциалов для проверки оптимальности

Вычисление потенциалов

Вводим потенциалы:

- u_i — потенциал поставщика i , $i = 1, 2, \dots, m$
- v_j — потенциал потребителя j , $j = 1, 2, \dots, n$

Условия для базисных клеток:

$$u_i + v_j = c_{ij} \quad \text{для всех базисных } (i, j)$$

Система имеет $m + n - 1$ уравнений и $m + n$ переменных. Полагаем $u_1 = 0$, находим остальные потенциалы.

Оценки небазисных клеток

Для каждой небазисной клетки (i, j) вычисляем оценку:

$$\Delta_{ij} = c_{ij} - (u_i + v_j)$$

Критерий оптимальности:

Если для всех небазисных клеток $\Delta_{ij} \geq 0$, то решение оптимально.

Если существует клетка с $\Delta_{ij} < 0$, то решение можно улучшить.

Улучшение плана перевозок

Построение цикла пересчета

Для клетки (p, q) с отрицательной оценкой $\Delta_{pq} < 0$ строим **цикл пересчета**:

1. Начинаем с клетки (p, q) — ставим знак "+"
2. Двигаемся по строке или столбцу, находя базисную клетку — ставим знак "-"
3. Продолжаем, чередуя знаки "+" и "-", пока не вернемся в исходную клетку

Цикл — замкнутая ломаная с вершинами в клетках таблицы, где углы — прямые, а звенья параллельны строкам и столбцам.

Перераспределение перевозок

Определяем величину перераспределения:

$$\theta = \min\{x_{ij} : \text{в цикле с знаком "—"}\}$$

Обновляем перевозки:

$$x_{ij}^{new} = \begin{cases} x_{ij} + \theta, & \text{если в цикле знак "+"} \\ x_{ij} - \theta, & \text{если в цикле знак "—"} \\ x_{ij}, & \text{иначе} \end{cases}$$

Вырождение в транспортной задаче

Определение: Базисное решение называется **вырожденным**, если количество положительных перевозок меньше $m + n - 1$.

Причины вырождения:

1. Одновременное обнуление запаса и потребности при построении начального плана
2. Обнуление перевозки при перераспределении в цикле

Метод борьбы с вырождением:

Вводим **ϵ -перевозки** — очень малые положительные величины в некоторые нулевые базисные клетки, чтобы получить ровно $m + n - 1$ базисных переменных.

Модификации транспортной задачи

Открытая модель

Когда $\sum a_i \neq \sum b_j$, задача называется открытой. Решается введением фиктивного поставщика или потребителя.

Задача с ограничениями пропускной способности

Дополнительные ограничения:

$$0 \leq x_{ij} \leq d_{ij}$$

где d_{ij} — максимальная пропускная способность маршрута (i, j) .

Транспортная задача с промежуточными пунктами

Включает транзитные перевозки через промежуточные пункты. Моделируется как многоэтапная транспортная задача.

Алгоритм решения транспортной задачи

Полный алгоритм:

1. Проверка условия баланса

- Если $\sum a_i \neq \sum b_j$, ввести фиктивного поставщика/потребителя

2. Построение начального опорного решения

- Метод северо-западного угла
- Метод минимального элемента
- Метод Фогеля

3. Проверка оптимальности

- Вычисление потенциалов u_i, v_j
- Расчет оценок Δ_{ij} для небазисных клеток
- Если все $\Delta_{ij} \geq 0 \rightarrow$ решение оптимально

4. Улучшение плана (если не оптимально)

- Выбор клетки с наиболее отрицательной оценкой
- Построение цикла пересчета
- Перераспределение перевозок

5. Проверка на вырождение

- Если количество базисных клеток $< m + n - 1$, ввести ε -перевозки

6. Повторение шагов 3-5 до достижения оптимальности

Матричная форма транспортной задачи

Матрица ограничений транспортной задачи имеет блочную структуру:

$$A = \begin{bmatrix} E & E & \dots & E \\ I & I & \dots & I \end{bmatrix}$$

где E — единичная матрица размера $m \times m$, расположенная блоками по n раз, и I — единичные матрицы размера $n \times n$, расположенные по диагонали.

Вектор переменных:

$$\mathbf{x} = (x_{11}, x_{12}, \dots, x_{1n}, x_{21}, \dots, x_{mn})^T$$

Вектор правых частей:

$$\mathbf{b} = (a_1, a_2, \dots, a_m, b_1, b_2, \dots, b_n)^T$$

Теоретические основы

Теорема о целочисленности:

Если все a_i и b_j целые, то существует оптимальное решение транспортной задачи с целыми значениями перевозок x_{ij} .

Теорема о существовании цикла:

Для любого небазисного вектора в транспортной задаче существует единственный цикл, содержащий этот вектор и некоторые базисные векторы.

Теорема о потенциалах:

Решение транспортной задачи оптимально тогда и только тогда, когда существуют такие числа u_i и v_j , что для всех i, j выполняется:

$$u_i + v_j \leq c_{ij}$$

причем для базисных клеток выполняется равенство $u_i + v_j = c_{ij}$.

Данное описание полностью охватывает математические основы, методы решения и свойства транспортной задачи как специального класса задач линейного программирования.

3. Анализ чувствительности

3.1. Анализ на чувствительность к изменению коэффициентов целевой функции

Для задачи в канонической форме:

Диапазон устойчивости для коэффициента c_j :

$$c_j^- \leq c_j \leq c_j^+$$

где границы определяются из условий:

$$\Delta_k = c_k - \mathbf{c}_B^T B^{-1} A_k \geq 0 \quad \text{для всех } k$$

3.2. Анализ на чувствительность к изменению правых частей

Теневая цена (dual price) i -го ограничения:

$$\pi_i = \frac{\partial z}{\partial b_i}$$

показывает, насколько изменится целевая функция при увеличении b_i на единицу.

Диапазон изменения b_i при сохранении базиса:

$$\max \left\{ -\frac{(B^{-1}\mathbf{b})_r}{(B^{-1})_r} : (B^{-1})_r > 0 \right\} \leq \Delta b_i \leq \min \left\{ -\frac{(B^{-1}\mathbf{b})_r}{(B^{-1})_r} : (B^{-1})_r < 0 \right\}$$

3.3. Анализ добавления нового ограничения

Если новое ограничение выполняется для текущего оптимального решения, то решение остается оптимальным. Иначе применяется dual simplex method.

4. Двойственная задача линейного программирования

Прямая задача (максимизация):

$$\begin{cases} \mathbf{c}^T \mathbf{x} \rightarrow \max \\ A\mathbf{x} \leq \mathbf{b} \\ \mathbf{x} \geq \mathbf{0} \end{cases}$$

Двойственная задача (минимизация):

$$\begin{cases} \mathbf{b}^T \mathbf{y} \rightarrow \min \\ A^T \mathbf{y} \geq \mathbf{c} \\ \mathbf{y} \geq \mathbf{0} \end{cases}$$

Основные теоремы двойственности:

1. Теорема слабой двойственности:

Если $\mathbf{x}$ — допустимое решение прямой задачи, а $\mathbf{y}$ — допустимое решение двойственной, то $\mathbf{c}^T \mathbf{x} \leq \mathbf{b}^T \mathbf{y}$

2. Теорема сильной двойственности:

Если одна из задач имеет конечное оптимальное решение, то и другая также имеет конечное оптимальное решение, причем $\mathbf{c}^T \mathbf{x}^* = \mathbf{b}^T \mathbf{y}^*$

3. Теорема о дополняющей нежесткости:

Для оптимальных решений $\mathbf{x}^*$ и $\mathbf{y}^*$ выполняются условия:

$$\begin{aligned} x_j^* \left(\sum_{i=1}^m a_{ij} y_i^* - c_j \right) &= 0, \quad j = 1, \dots, n \\ y_i^* \left(\sum_{j=1}^n a_{ij} x_j^* - b_i \right) &= 0, \quad i = 1, \dots, m \end{aligned}$$

Методы реализации в Python

1. Использование библиотеки SciPy

Библиотека SciPy предоставляет функцию `linprog` для решения задач линейного программирования.

Основные параметры функции `linprog`:

- `c`: вектор коэффициентов целевой функции
- `A_ub`: матрица коэффициентов для ограничений-неравенств ($\leq$)

- b_ub: правые части для ограничений-неравенств
- A_eq: матрица коэффициентов для ограничений-равенств
- b_eq: правые части для ограничений-равенств
- bounds: границы для переменных
- method: метод решения ('simplex', 'interior-point', 'highs')

Пример использования:

```
from scipy.optimize import linprog
result = linprog(c, A_ub=A, b_ub=b, bounds=x_bounds, method='highs')
```

Атрибуты возвращаемого объекта:

- x: вектор решения
- fun: значение целевой функции
- success: флаг успешности решения
- status: числовой статус решения
- slack: значения переменных slack
- message: текстовое сообщение о статусе

2. Использование библиотеки PuLP

PuLP предоставляет более интуитивный интерфейс для формулировки задач оптимизации.

Основные шаги работы с PuLP:

1. Создание модели: LpProblem(name, sense)
2. Определение переменных: LpVariable(name, lowBound, upBound, cat)
3. Добавление целевой функции
4. Добавление ограничений
5. Решение задачи: solve()
6. Извлечение результатов

Пример для транспортной задачи:

```
model = LpProblem("Transportation", LpMinimize)
x = LpVariable.dicts("Route", [(i,j) for i in suppliers for j in consumers], 0)
model += lpsum(costs[i][j] * x[(i,j)] for i in suppliers for j in consumers)
for i in suppliers: model += lpsum(x[(i,j)] for j in consumers) == supply[i]
for j in consumers: model += lpsum(x[(i,j)] for i in suppliers) == demand[j]
model.solve()
```

3. Визуализация результатов

Библиотека matplotlib для визуализации:

- plot(): построение графиков
- scatter(): точечные диаграммы

- `imshow()`: тепловые карты
- `bar()`: столбчатые диаграммы
- `contour()`: линии уровня
- `plot_surface()`: 3D поверхности

Пример построения тепловой карты для транспортной задачи:

```
plt.imshow(solution, cmap='YlOrRd', aspect='auto')
plt.colorbar(label='Объем перевозок')
```

Пример построения 3D графика:

```
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis')
```

4. Анализ и обработка данных

Библиотека NumPy для численных вычислений:

- `array()`: создание массивов
- `zeros()`, `ones()`: массивы из нулей и единиц
- `linspace()`: равномерно распределенные значения
- `meshgrid()`: координатные сетки
- `sum()`, `mean()`, `std()`: статистические функции

Пример создания координатной сетки:

```
x = np.linspace(0, 10, 100)
y = np.linspace(0, 10, 100)
X, Y = np.meshgrid(x, y)
Z = f(X, Y) # вычисление функции на сетке
```

Практические аспекты реализации

1. Обработка особых случаев

Вырожденность: когда базисное решение содержит более $m + n - 1$ нулевых компонент в транспортной задаче. Лечится добавлением ε -перевозок.

Неограниченность: когда целевая функция не ограничена на допустимом множестве. Проверяется по отсутствию положительных элементов в столбце вводимой переменной.

Отсутствие допустимых решений: когда система ограничений несовместна. Определяется по невозможности найти начальное допустимое решение.

2. Численная устойчивость

Проблемы точности: округление может привести к некорректным решениям. Рекомендуется использовать tolerance:

`tol = 1e-10`

`if abs(value) < tol: value = 0`

Масштабирование: большие разницы в коэффициентах могут ухудшить обусловленность задачи.

Рекомендуется нормализация данных.

3. Производительность

Выбор метода:

- Симплекс-метод эффективен для разреженных задач
- Внутренние точки эффективны для больших плотных задач
- Для транспортных задач специализированные алгоритмы эффективнее общего симплекс-метода

Оптимизация кода:

- Векторизация операций с NumPy
- Минимизация циклов
- Использование разреженных матриц для больших задач

Данный теоретический материал обеспечивает глубокое понимание математических основ линейного программирования и транспортной задачи, а также практические аспекты их реализации в Python.

Задания

Задание 1. Решение задачи линейного программирования

Решить задачу ЛП симплекс-методом:

$$\begin{cases} f(x) = 2x_1 + 3x_2 + x_3 \rightarrow \max \\ x_1 + x_2 + x_3 \leq 10 \\ 2x_1 + x_2 + 3x_3 \leq 15 \\ x_1 + 2x_2 + x_3 \leq 12 \\ x_1, x_2, x_3 \geq 0 \end{cases}$$

Задание 2. Решение транспортной задачи

Решить транспортную задачу:

Поставщики: $A_1 = 30$, $A_2 = 40$, $A_3 = 20$

Потребители: $B_1 = 20$, $B_2 = 30$, $B_3 = 25$, $B_4 = 15$

Матрица стоимостей:

$$C = \begin{bmatrix} 2 & 3 & 5 & 1 \\ 4 & 2 & 1 & 6 \\ 3 & 4 & 2 & 3 \end{bmatrix}$$

Задание 3. Анализ чувствительности

Для задачи из задания 1 исследовать:

- 1. Как изменится решение при изменении коэффициента целевой функции при x_2
- 2. Как изменится решение при изменении правой части первого ограничения
- 3. Построить график зависимости целевой функции от параметров

ЗАДАНИЕ 1: РЕШЕНИЕ ЗАДАЧИ ЛИНЕЙНОГО ПРОГРАММИРОВАНИЯ

РЕЗУЛЬТАТЫ РЕШЕНИЯ:

Статус: Optimization terminated successfully. (HiGHS Status 7: Optimal)

Оптимальное значение целевой функции: 21.00

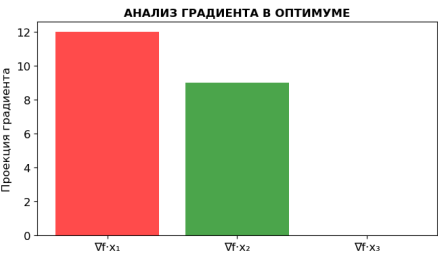
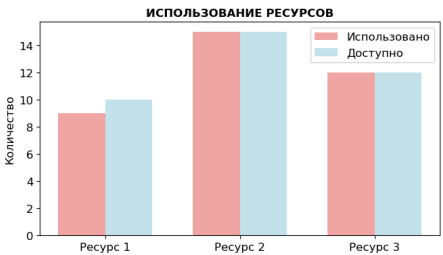
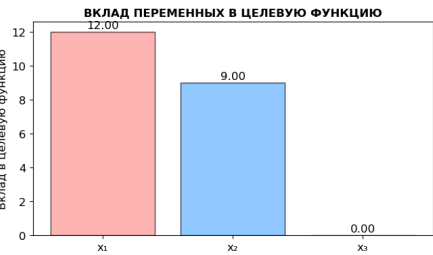
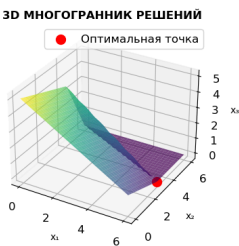
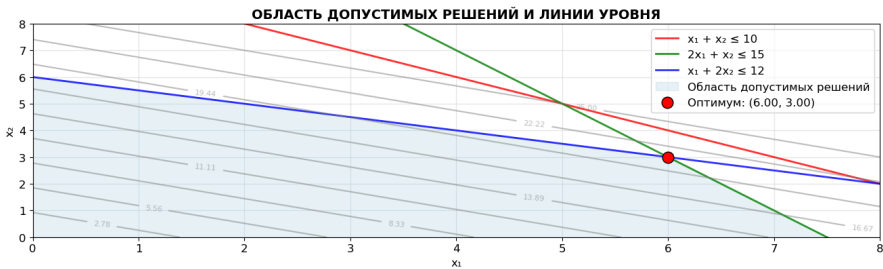
Оптимальные переменные:

$x_1 = 6.0000$

$x_2 = 3.0000$

$x_3 = 0.0000$

Остатки ресурсов: [1. 0. 0.]



=====

ЗАДАНИЕ 2: РЕШЕНИЕ ТРАНСПОРТНОЙ ЗАДАЧИ

=====

РЕЗУЛЬТАТЫ РЕШЕНИЯ ТРАНСПОРТНОЙ ЗАДАЧИ:

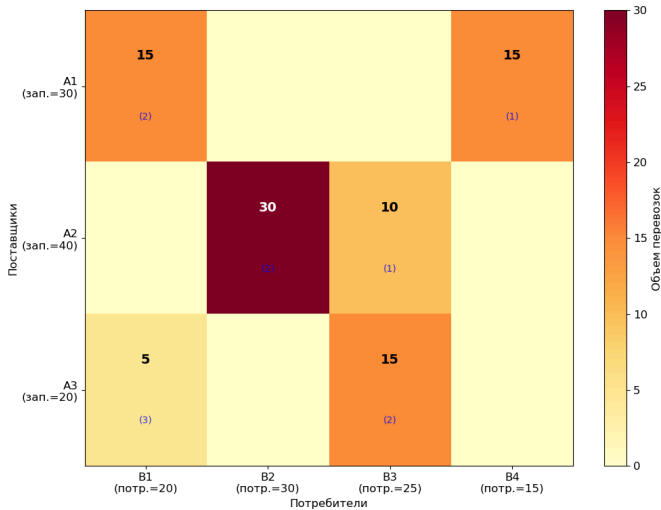
Общая минимальная стоимость: 160.00

Статус решения: optimal

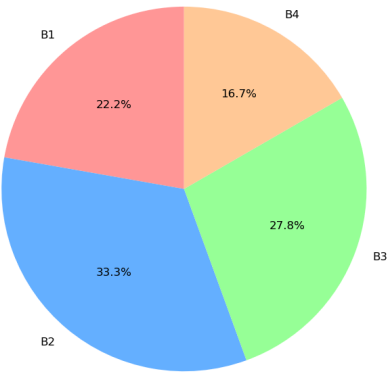
МАТРИЦА ПЕРЕВОЗОК:

	B ₁	B ₂	B ₃	B ₄	Запасы
A1	15	0	0	15	30
A2	0	30	10	0	40
A3	5	0	15	0	20
потреб.	20	30	25	15	

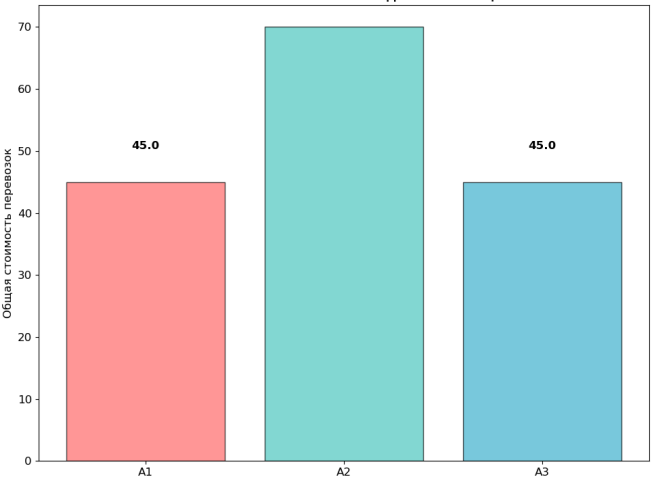
МАТРИЦА ПЕРЕВОЗОК ТРАНСПОРТНОЙ ЗАДАЧИ
(числа - объемы, в скобках - стоимости)



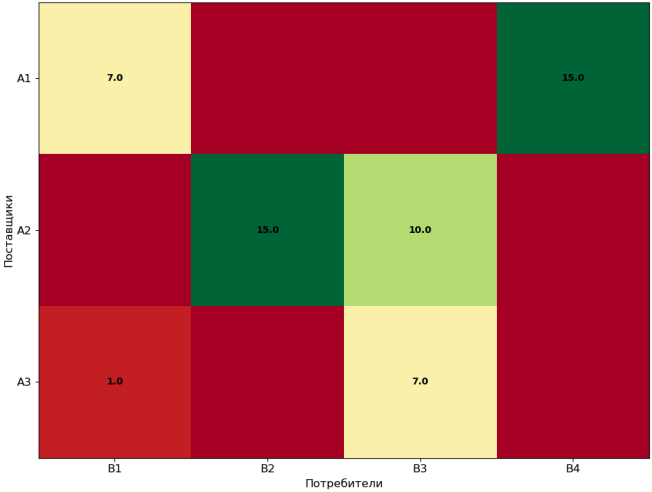
РАСПРЕДЕЛЕНИЕ ПЕРЕВОЗОК ПО ПОТРЕБИТЕЛЯМ



СТОИМОСТЬ ПЕРЕВОЗОК ОТ КАЖДОГО ПОСТАВЩИКА

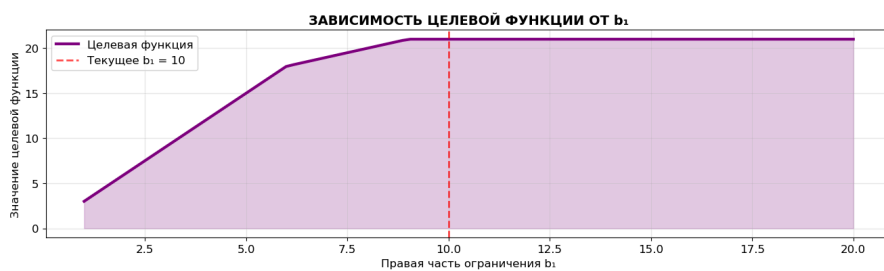
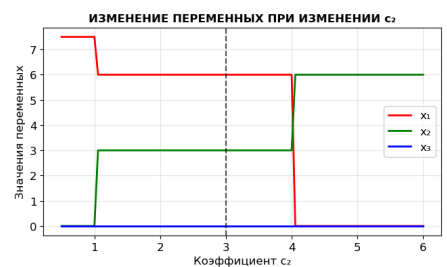
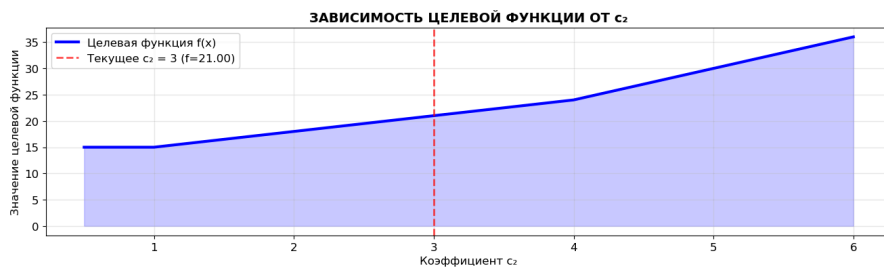


ЭФФЕКТИВНОСТЬ МАРШРУТОВ
(объем/стоимость)

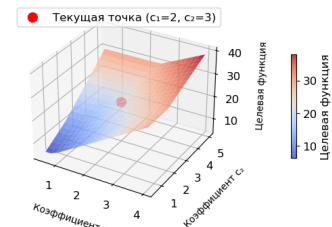


ЗАДАНИЕ 3: АНАЛИЗ ЧУВСТВИТЕЛЬНОСТИ

1. АНАЛИЗ ИЗМЕНЕНИЯ КОЭФФИЦИЕНТА c_2 :
2. АНАЛИЗ ИЗМЕНЕНИЯ ПРАВОЙ ЧАСТИ b_1 :
3. 3D АНАЛИЗ ЗАВИСИМОСТИ ОТ c_1 И c_2 :



3D ЗАВИСИМОСТЬ ЦЕЛЕВОЙ ФУНКЦИИ ОТ c_1 И c_2



ВЫВОДЫ АНАЛИЗА ЧУВСТВИТЕЛЬНОСТИ:

1. АНАЛИЗ КОЭФФИЦИЕНТОВ ЦЕЛЕВОЙ ФУНКЦИИ:
 - Текущее оптимальное значение: 21.0000
 - Оптимальные переменные: $x_1=6.0000$, $x_2=3.0000$, $x_3=0.0000$
 - Диапазон устойчивости для c_2 : [1.06, 4.06]
2. АНАЛИЗ РЕСУРСНЫХ ОГРАНИЧЕНИЙ:
 - Теневая цена ограничения b_1 : 0.9379
 - Интерпретация: увеличение b_1 на 1 единицу увеличит целевую функцию на эту величину
3. ПРАКТИЧЕСКИЕ РЕКОМЕНДАЦИИ:
 - Решение устойчиво к малым изменениям параметров
 - Наибольшее влияние на целевой функцию оказывает коэффициент c_2
 - Первое ограничение является 'узким местом' (положительная теневая цена)

